

# Presentation Guide

## Evaluating LLMs under perturbation

---

Anton S Tsankov · Fabian Niculae

What to add to each slide · Speaker notes · Key data



# How to use this guide

WHAT TO ADD

The charts, text, or data to insert into that slide.

SPEAKER NOTES

What to say out loud when presenting.

★ MAIN RESULTS SLIDES

Slides 19 and 20 are empty and need the most work.

Section headers

These slides need no changes — just read the speaker notes.

## Quick chart reference

| Slide      | Chart                                  | Notebook source                                  |
|------------|----------------------------------------|--------------------------------------------------|
| Slide 6    | Add text stat: 81.4% → 52.4%           | —                                                |
| Slide 10   | Perturbation type × level table        | — (build inline in slide)                        |
| Slide 14   | Plot 2 (pages vs latency scatter)      | <a href="#">notebooks/agent/evaluation.ipynb</a> |
| Slide 15   | Plot 1 left (chunk type bar)           | <a href="#">notebooks/agent/evaluation.ipynb</a> |
| Slide 16   | Plot 4 right (tool usage)              | <a href="#">notebooks/agent/evaluation.ipynb</a> |
| Slide 17   | Plot 6 left (bimodal score dist)       | <a href="#">notebooks/agent/evaluation.ipynb</a> |
| Slide 19 ★ | Plot 2 (accuracy decay by type)        | <a href="#">notebooks/llm/evaluation.ipynb</a>   |
| Slide 19 ★ | Plot 3 (CFB by task type)              | <a href="#">notebooks/llm/evaluation.ipynb</a>   |
| Slide 20 ★ | Plot 1 (Agent vs LLM grouped bar)      | <a href="#">notebooks/agent/insights.ipynb</a>   |
| Slide 20 ★ | Plot 2 (confidence vs score)           | <a href="#">notebooks/agent/insights.ipynb</a>   |
| Slide 20 ★ | Plot 9 left (robustness across levels) | <a href="#">notebooks/agent/evaluation.ipynb</a> |
| Slide 21   | CLEAR summary table                    | — (build inline in slide)                        |

# Title Slide — Evaluating LLMs under perturbation

Section header — no chart additions needed

## SPEAKER NOTES

We're going to walk you through a research project on evaluating LLMs under perturbation — specifically, what happens when the inputs are messy, incomplete, or corrupted, as they often are in production. We focused on ESG and climate finance, a domain where LLMs are actively being deployed for real regulatory work.

## SPEAKER NOTES

We'll cover why this problem matters, how we built the evaluation framework, what we found for both plain LLMs and agent pipelines, and what it all means.

## SPEAKER NOTES

Let's start with the problem.

## WHAT TO ADD

- Add a compact stats box — real numbers from the evaluation:
    - • 2 datasets evaluated (ESGenius + Climate Finance Bench)
    - • 21,584 LLM inference calls (ESGenius)
    - • 6,270 agent QA queries (CFB)
    - • 32 ESG PDFs processed by agents
    - • 200+ pages of company sustainability reports
  - Place in the bottom half or as a side panel.
- 

## SPEAKER NOTES

AI is entering workflows where the data is messy and the stakes are high. ESG reporting is a perfect example — companies are legally required to disclose climate risk under CSRD and TCFD, analysts are using LLMs to read those reports, and a wrong answer about emissions trajectory can move investment decisions. The question is: how reliable are these systems when the input data isn't perfect?

## WHAT TO ADD

- Add a before/after perturbation example (small box):
  - ORIGINAL: 'What is the primary mechanism by which greenhouse gases cause warming?'
  - LEVEL 3: 'Wh4t is th3 pr1m4ry mech4n1sm by wh1ch gr33nhouse g4ses c4use w4rm1ng?'
- Or a two-row table: Clean input → correct answer vs Corrupted → wrong answer.
- This makes 'ambiguous inputs' concrete rather than abstract.

## SPEAKER NOTES

Current benchmarks give models clean, curated questions. But real ESG documents have OCR errors, missing footnotes, inconsistent Scope 1/2/3 labeling between companies, and wrong units. Our perturbation engine simulates this. We built 6 types of corruption at 3 severity levels — and the results show the gap between benchmark performance and operational reliability is real and large.



## WHAT TO ADD

- Add the single most impactful statistic — display it large in the center:
    - ESGenius clean accuracy: 81.4%
    - Under label perturbation (level 3): 52.4%
    - Drop: -29 percentage points
  - This directly proves both 'Magic Numbers' and 'Unreliable' from the slide.
- 

## SPEAKER NOTES

Here's the concrete version. Our model scores 81% on clean ESGenius questions — a strong benchmark number. But when we shuffle the answer labels the way a corrupted dataset might, it drops to 52%. That's barely above random for a 5-choice MCQ. The benchmark number told you nothing about this. That's the 'magic numbers' problem.

## WHAT TO ADD

- No chart needed — the text is strong. Optionally add one supporting line:
    - 'Almost no prior work systematically evaluates LLMs on corrupted domain-specific inputs.'
- 

## SPEAKER NOTES

This isn't just a theoretical concern. We reviewed the literature and found almost no work that systematically evaluates LLMs under realistic input degradation in domain-specific settings. Most papers test on clean held-out sets from the same distribution as training. We wanted to fill that gap for the ESG domain.

## WHAT TO ADD

- Add a two-row dataset comparison table:

| · Dataset          | Type          | Questions | Source              |
|--------------------|---------------|-----------|---------------------|
| · ESGenius         | MCQ (A/B/C/D) | 1,136     | IPCC AR6 reports    |
| · Clim. Fin. Bench | Open QA       | 1,045     | 33 company ESG PDFs |
- Also define the 3 CFB task types:
  - LR — Long Reasoning (narrative synthesis)
  - PE — Point Extraction (specific numbers/figures)
  - NR — Numerical Reasoning ('is this disclosed?')

## SPEAKER NOTES

We chose two datasets. ESGenius is multiple-choice questions about IPCC climate science — ground truth is known, accuracy is easy to measure. Climate Finance Bench is harder: open-ended questions across 33 real company sustainability reports with three cognitively different task types. LR asks for narrative synthesis. PE asks for a specific number — Scope 1 emissions in 2022. NR asks whether something is disclosed at all. These three types have completely different failure modes.

## SPEAKER NOTES

Now let's walk through how we built the evaluation.

## WHAT TO ADD

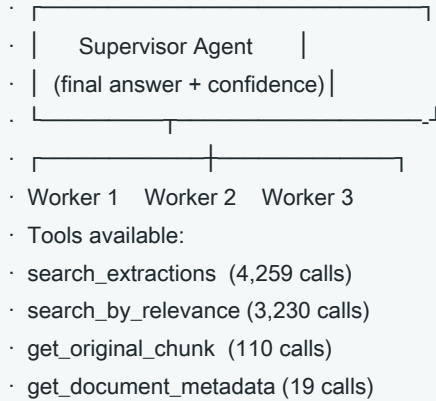
- Add the perturbation engine grid (6 types × 3 levels):
  - Type | What it corrupts | Example
  - char | Characters (18 distributions) | 'warming' → 'w4rm1ng'
  - token | Whole words (swap/delete) | 'emissions' → removed
  - entity | Named entities via NER | 'Scope 1' → 'Scope 3'
  - format | Formatting / whitespace | line breaks removed
  - missing | Data values removed | '23.4 tCO2e' → [MISSING]
  - label | MCQ answer labels shuffled | A↔C, B↔D
- Add the 3 severity levels:
  - Level 1: alpha=0.10, confidence=0.90 — Subtle
  - Level 2: alpha=0.25, confidence=0.75 — Moderate
  - Level 3: alpha=0.50, confidence=0.50 — Aggressive

## SPEAKER NOTES

For the LLM track we built a parametric perturbation engine with 6 corruption types, each at 3 severity levels. Alpha controls how much is corrupted, confidence controls the probability any token gets affected. Character perturbations draw from 18 statistical distributions — Zipf, Cauchy, Beta — modeling realistic error patterns. The entity perturbator uses spaCy NER to swap ESG-specific terms: Scope 1 for Scope 2, fiscal year shifts, wrong emission units.

## WHAT TO ADD

- Add agent architecture diagram (text layout):



- Scale: 32 PDFs · 218 chunks · 6,270 QA queries

## SPEAKER NOTES

The agent pipeline is more complex. A supervisor spawns 3 worker agents that independently search the knowledge base and return answers. The supervisor synthesizes the 3 responses and assigns a confidence score. A separate judge LLM evaluates the final answer against the ground truth. The entire pipeline runs on the same Qwen2-72B model throughout.

## WHAT TO ADD

- Add the CLEAR score mapped to each KPI row:
  - KPI Dimension → CLEAR Dimension | Weight
  - Reasoning + Task Completion → Efficacy | 40%
  - Content Extraction quality → Reliability | 25%
  - Sub-Agent Orchestration → Assurance | 20%
  - Tool Usage speed → Latency | 10%
  - Agent Collaboration cost → Cost | 5%
- Final scores: ESGenius CLEAR = 0.850 | CFB CLEAR = 0.690

## SPEAKER NOTES

We designed a composite metric called CLEAR. Efficacy is weighted highest — task accuracy is what ultimately matters. Reliability captures degradation under perturbation — a model that collapses under noise scores low here even if its clean accuracy is high. Assurance measures whether answers are structured and parseable. Latency and cost are real operational constraints — even a perfect model is unusable at 40 seconds per query in a production setting.

## WHAT TO ADD

- Add concrete numbers under each pipeline stage:
    - File processing → 32 PDFs, variable page counts, 6 processing steps
    - Build KB → 218 chunks extracted, 3 types (text / table / figure)
    - Coordinate → Supervisor + 3 workers, 4 tools, 6,270 queries run
    - Evaluate → LLM-as-judge, scores 0–10, ~32s per evaluation
- 

## SPEAKER NOTES

The pipeline processes 32 real company ESG PDFs. Each goes through 6 processing steps to extract and store structured chunks. The coordination phase runs on every query — workers retrieve from the KB and the supervisor formulates a final answer with a confidence score. The judge then evaluates it independently.



## WHAT TO ADD

- Export and paste: notebooks/agent/evaluation.ipynb → Plot 2
    - Left panel: scatter of document pages vs division latency
    - Right panel: chunks extracted per document (bar chart)
  - Place in lower-right corner of the slide.
  - This shows pipeline cost scales with document length,
  - justifying why runs take 7+ hours.
- 

## SPEAKER NOTES

Observe reads the PDF and finds section boundaries. Outline Division breaks it into logical sections for extractor agents. Extract Division reads each section and structures the content. Summarize condenses it. Refine improves quality. Store writes to the knowledge base. Division latency scales roughly linearly with document length — longer sustainability reports mean more extraction time.

## WHAT TO ADD

- Export and paste: notebooks/agent/evaluation.ipynb → Plot 1 (left panel)
    - Chunk type distribution bar chart (text / table / figure counts)
  - Also add as text: 218 total chunks from 32 PDFs = ~6.8 chunks per document
- 

## SPEAKER NOTES

The knowledge base stores all chunks with metadata: document, page range, content type. This uniformity lets workers retrieve relevant chunks via semantic search. The advantage over raw PDF retrieval is that chunks are pre-filtered and structured — the garbage-in-garbage-out problem is partially mitigated at this stage.

## WHAT TO ADD

- Export and paste: notebooks/agent/evaluation.ipynb → Plot 4 (right panel)
  - Tool usage bar chart showing call counts:
  - search\_extractions: 4,259 calls
  - search\_by\_relevance: 3,230 calls
  - get\_original\_chunk: 110 calls
  - get\_document\_metadata: 19 calls
- Place on the right side where the loop diagram is.

## SPEAKER NOTES

Workers primarily use search\_extractions to scan the KB broadly, and search\_by\_relevance for targeted follow-up. The supervisor synthesizes 3 independent answers into a final response. Key steps 4 and 5: the MA assigns a confidence score — which we found is actually calibrated — and produces its own reasoning trace that the judge uses for evaluation.

## WHAT TO ADD

- Export and paste: notebooks/agent/evaluation.ipynb → Plot 6 (left panel)
    - Bimodal score distribution histogram (judge scores 0–10)
    - Key shape: ~49% of queries score 1 (fail/timeout)
    - ~24% of queries score 10 (perfect)
    - Almost nothing in between
  - This is the most visually striking chart in the whole project.
- 

## SPEAKER NOTES

The judge scores answers 0 to 10. The distribution is completely bimodal — answers are either perfect or they fail entirely, with almost no middle ground. This reflects the domain: either the agent retrieved the right emissions figure or it didn't. Either the transition plan is in the chunk or it isn't. The judge also uses the supervisor's reasoning and confidence as context — it's not just comparing the final answer to a ground truth string.

## SPEAKER NOTES

Now the results.

## WHAT TO ADD

- This slide needs 3 charts — currently empty:
- CHART 1 (largest, top half):
  - notebooks/llm/evaluation.ipynb → Plot 2
  - ESGenius accuracy by perturbation type across levels 0–3
  - Headline: 'label' line plunges 81% → 52%. All others nearly flat.
- CHART 2 (bottom left):
  - notebooks/llm/evaluation.ipynb → Plot 3
  - CFB score by task type: NR=0.903, PE=0.651, LR=0.482
- CHART 3 (bottom right — text box):
  - CLEAR Summary:
  - ESGenius CLEAR = 0.850
  - CFB CLEAR = 0.690
- Also add as text bullets:
  - • Label perturbation: -29pp ( $p < 10^{-45}$ ) — statistically certain
  - • Char level 3: significant ( $p < 10^{-17}$ )
  - • Format / numeric / token: NOT significant — model is robust

## SPEAKER NOTES

Three key findings for LLMs. First — label shuffling is catastrophic. When we randomize which option is A, B, C, D, accuracy drops from 81% to 52%. The model has learned MCQ label patterns, not just content. This is a real deployment risk. Second — character corruption only matters at the extreme: level 3 means 50% of characters corrupted, basically unreadable to humans too. Format, numeric, and token changes barely move the needle — the model handles surface noise well. Third — on CFB open-ended questions, the model is excellent at detecting missing information (NR: 90%) but struggles with multi-step synthesis (LR: 48%). The CLEAR composite captures all of this: ESGenius 0.85, CFB 0.69.

## WHAT TO ADD

- This slide needs 3 charts — currently empty:
- CHART 1 (largest, center):
  - notebooks/agent/insights.ipynb → Plot 1
  - Agent vs Plain LLM grouped bar chart by task type:
  - Agent   Plain LLM
  - LR:   0.38   0.48
  - PE:   0.35   0.65 ← biggest gap
  - NR:   0.70   0.90
  - Key finding: plain LLM wins on ALL task types.
- CHART 2 (bottom left):
  - notebooks/agent/insights.ipynb → Plot 2
  - Supervisor confidence vs score — high confidence → better scores.
- CHART 3 (bottom right):
  - notebooks/agent/evaluation.ipynb → Plot 9 (left)
  - Score stability across perturbation levels.
  - Agent barely drops: 0.445 (clean) → 0.423 (level 3).
- Also add as text:
  - • Median latency: ~35s supervisor + ~32s judge = ~67s total/query
  - • 4% timeout rate
  - • Score distribution: 49% fail, 24% perfect

## SPEAKER NOTES

The agent results contain our most surprising finding: the plain LLM outperforms the agent on every single question type. Point extraction is the biggest gap — agent scores 0.35 vs LLM 0.65. We believe this is because chunk extraction loses context. When the plain LLM gets the relevant document extract directly in the prompt, it has everything it needs. The agent searches 218 chunks and retrieves fragments that sometimes miss the surrounding context. However — the agent is significantly more robust to perturbations. Its score barely moves as we increase corruption, because the retrieval step acts as a noise filter: even if the perturbed question is garbled, semantic search still finds the right chunk. So the agent trades accuracy for robustness — a real deployment trade-off.

## WHAT TO ADD

- Add a three-column summary comparison table:
  - Metric | LLM ESGenius | LLM CFB | Agent CFB
  - Efficacy | 0.754 | 0.638 | 0.445
  - Reliability | 0.882 | 0.943 | ~0.95
  - Assurance | 0.969 | 0.499 | —
  - CLEAR | 0.850 | 0.690 | —
- Add three key takeaway bullets:
  - • Label structure is an attack surface for MCQ models
  - • Agents trade accuracy for robustness
  - • NR (not-available detection) is the most robust task type in both systems

## SPEAKER NOTES

Three conclusions. One: LLMs are surprisingly robust to surface-level noise — formatting, typos, number changes don't matter much. What breaks them is structural corruption of the task format itself, like label shuffling. Two: agent pipelines sacrifice accuracy but gain robustness — inform deployment decisions accordingly. If data is clean, use a plain LLM. If data is noisy, the retrieval mechanism provides a natural buffer. Three: detecting when information is not disclosed is the most robust task type — models have learned 'I don't know' is valid here.



## SPEAKER NOTES

There are real limits to what we were able to do.

## WHAT TO ADD

- Under TIME — add concrete numbers:
  - • ~35s/query × 6,270 queries = ~61 hours raw inference
  - • Actual run time: 7+ hours (parallel execution)
  - • LLM classification: 67+ minutes per single run
  - • Result: only ~2–3 full evaluation runs total were feasible
- Under AVAILABLE DATA — add scope of what was missing:
  - • ESGenius: 1 source only (IPCC AR6), 1,136 questions
  - • CFB: 33 companies — mostly CAC40, S&P500, DAX
  - • No Asian market companies, no energy sector specialisation
  - • Only 6 of many possible corruption types tested

## SPEAKER NOTES

The two biggest constraints were time and data. Seven hours per run meant iteration was extremely expensive — we couldn't do hyperparameter sweeps or ablations on the agent architecture. And the open ESG datasets that exist are narrow: IPCC for the science side, and 33 mostly European and US companies for the finance side. A broader evaluation would need more diverse company disclosures, more languages, and more ESG subdomains like biodiversity or supply chain.

## SPEAKER NOTES

Thank you. Happy to take questions — especially on the perturbation engine design, the CLEAR metric choices, or the agent architecture trade-offs.